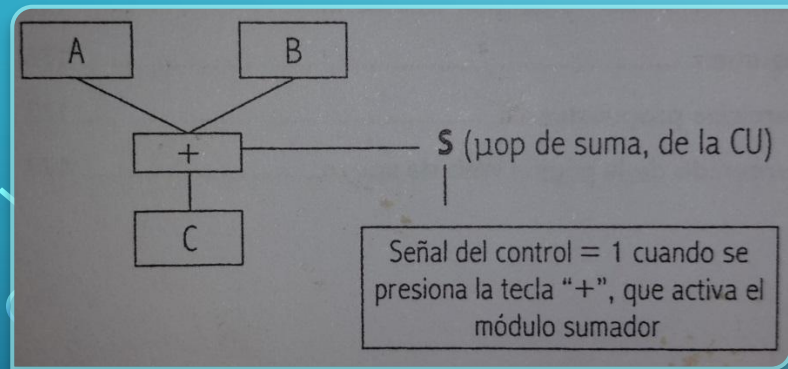


UNIDAD 3

DISEÑO DE UNA COMPUTADORA DIGITAL

DISEÑO DE UNA COMPUTADORA DIGITAL



- El diseño de una computadora digital es la organización de módulos de HW relacionados por rutas de control y datos. Su función es permitir el flujo de señales binarias para transformar datos de entrada en información útil al usuario de la computadora
- Cada módulo realiza una o varias operaciones sobre datos codificados en sistema binario.
- El diseño de una computadora es más abstracto que el diseño de la lógica de un módulo y se ocupa de ensamblarlos.

DISEÑO DE UNA COMPUTADORA DIGITAL

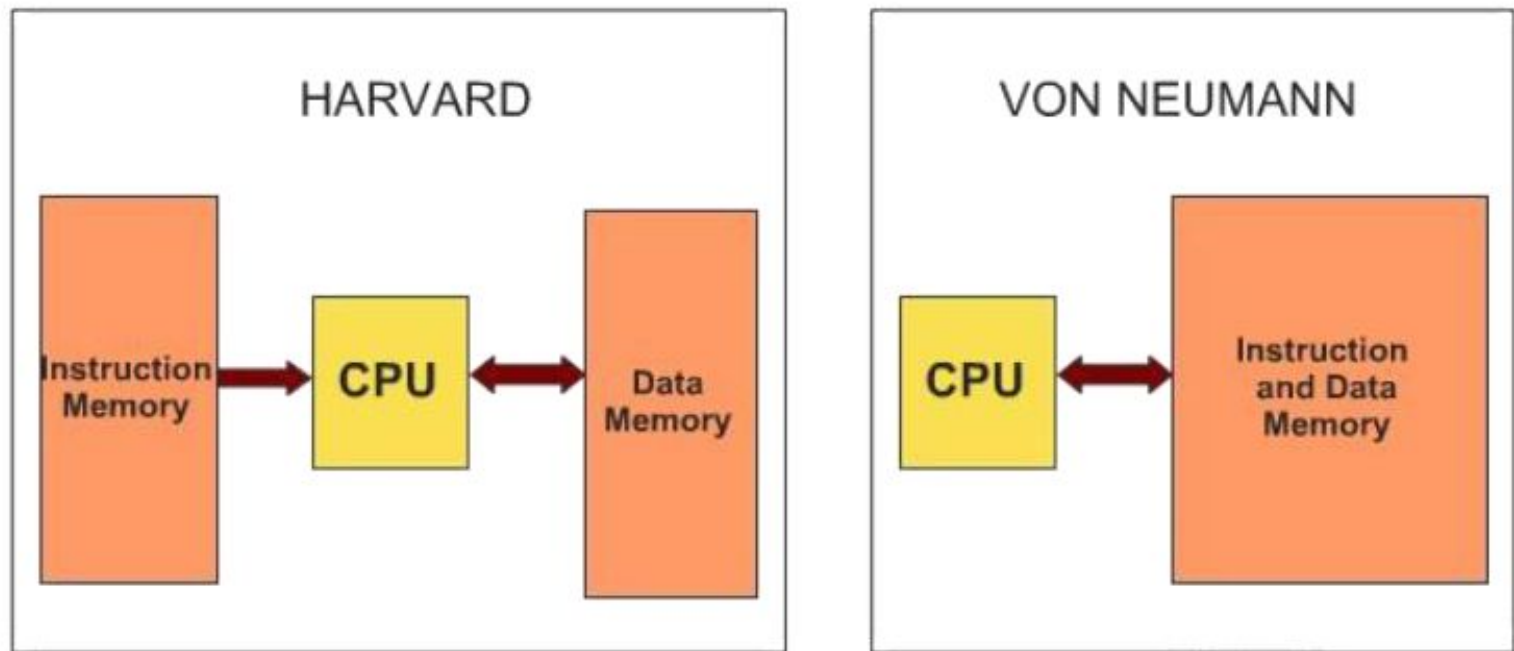


Figura modificada de "Fundamentals of Microcontrollers" de John Donovan (NXP)

DISEÑO DE UNA COMPUTADORA DIGITAL

Arquitectura Harvard vs von Neumann

- **Arquitectura Harvard**

- Memorias de código y datos
 - espacio de memoria diferentes
 - caminos separados (exclusivos) hacia la CPU
- Más rápida

- **Arquitectura von Neumann**

- Código y datos mapeados en un solo espacio
- Más flexible:
 - Fácilmente puede escribir en memoria de código

RELACIÓN ENTRE EL DISEÑO DEL HW Y LA EJECUCIÓN DE INSTRUCCIONES

- La programación en lenguaje de máquina implica el conocimiento del tipo de instrucciones en **código de máquina** (o **código nativo**) a las que obedece la computadora. Para simplificar la tarea de recordar secuencias binarias tediosas a la hora de programar, se ha desarrollado un lenguaje simbólico de bajo nivel conocido como **Assembler**.
- Cada computadora (y sus compatibles) tiene su propio **Assembler**, que tiene una íntima relación con su diseño.

RELACIÓN ENTRE EL DISEÑO DEL HW Y LA EJECUCIÓN DE INSTRUCCIONES

- Si en una computadora se ingresa un programa en lenguaje simbólico de alto nivel que indique:

BEGIN

Input A

Input B

$C = A + B$

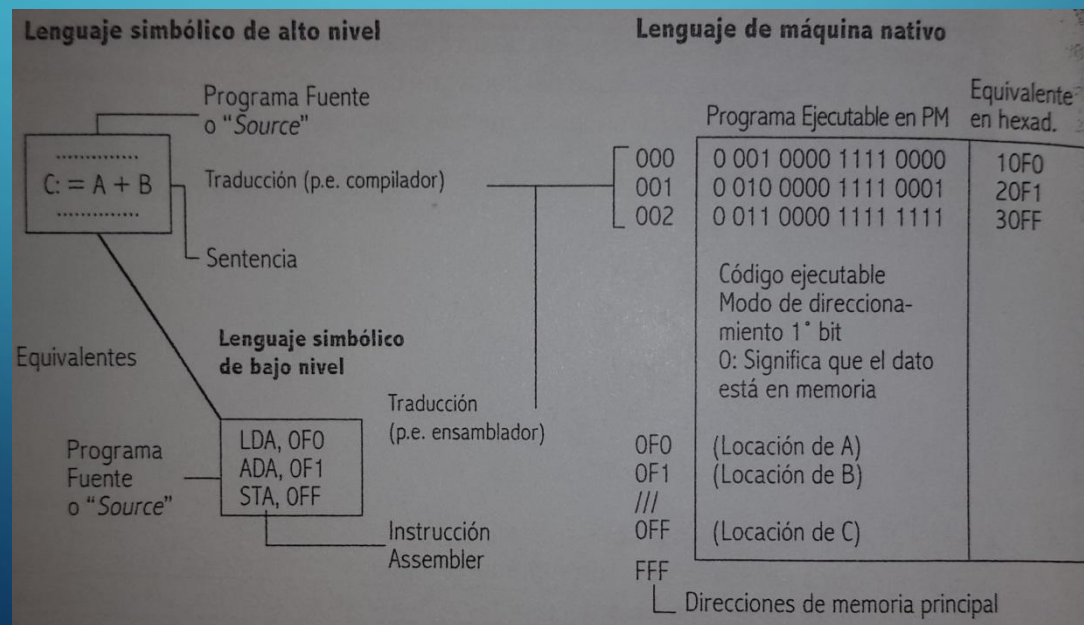
Output C

END

- Este programa será convertido en programa ejecutable con la ayuda de un programa “traductor”, donde para cada sentencia del programa fuente (o *source*) corresponderán n instrucciones en lenguaje de máquina.
- $C = A + B$ en Assembler equivale a 3 instrucciones: LDA, ADA, STA
- Assembler tiene una relación 1 a 1 con las instrucciones en código de máquina

RELACIÓN ENTRE EL DISEÑO DEL HW Y LA EJECUCIÓN DE INSTRUCCIONES

- La programación en lenguaje de máquina permite aumentar las prestaciones del sistema operativo, por ejemplo, al crear rutinas no definidas, que no podrían programarse en los lenguajes de alto nivel.



INSTRUCCIONES

- Cuando la computadora realiza una tarea compleja, a pedido del usuario, ejecuta una serie de pasos simples representados por su propio juego de instrucciones. Estas instrucciones constituyen su lenguaje de máquina o nativo.
- No es usual que el programador plantee la tarea en términos de secuencias binarias, sino que utiliza un lenguaje simbólico más orientado a su modalidad de expresión que a la de la computadora.
- Sin embargo, todo programa que utiliza un lenguaje simbólico debe traducirse a código de máquina antes de su ejecución

INSTRUCCIONES

- El **código de instrucción** es la combinación de bits que la unidad de control de la CPU interpreta para generar microoperaciones que permitan su ejecución.
- El formato de instrucción más simple es el que asigna un grupo de bits para representar una “acción” y otro grupo para representar el “dato” al que afecta esta acción.



P

- La cantidad de bits del COP (u OPCODE) determina el número de acciones distintas que se podrían definir, donde n bits determinan 2^n códigos de operaciones distintos.
- Por lo general, el segundo grupo de bits hace referencia a un dato en memoria, por lo tanto, determina la dirección de la posición de memoria donde se aloja el dato. La cantidad de bits debe permitir hacer referencia a cualquier posición de memoria.

INSTRUCCIONES

- La memoria de lectura/escritura está dividida lógicamente en memoria asignada a programa y memoria asignada a datos y constituye el modo de almacenamiento de la computadora.

PROCESAMIENTO DE INSTRUCCIÓN

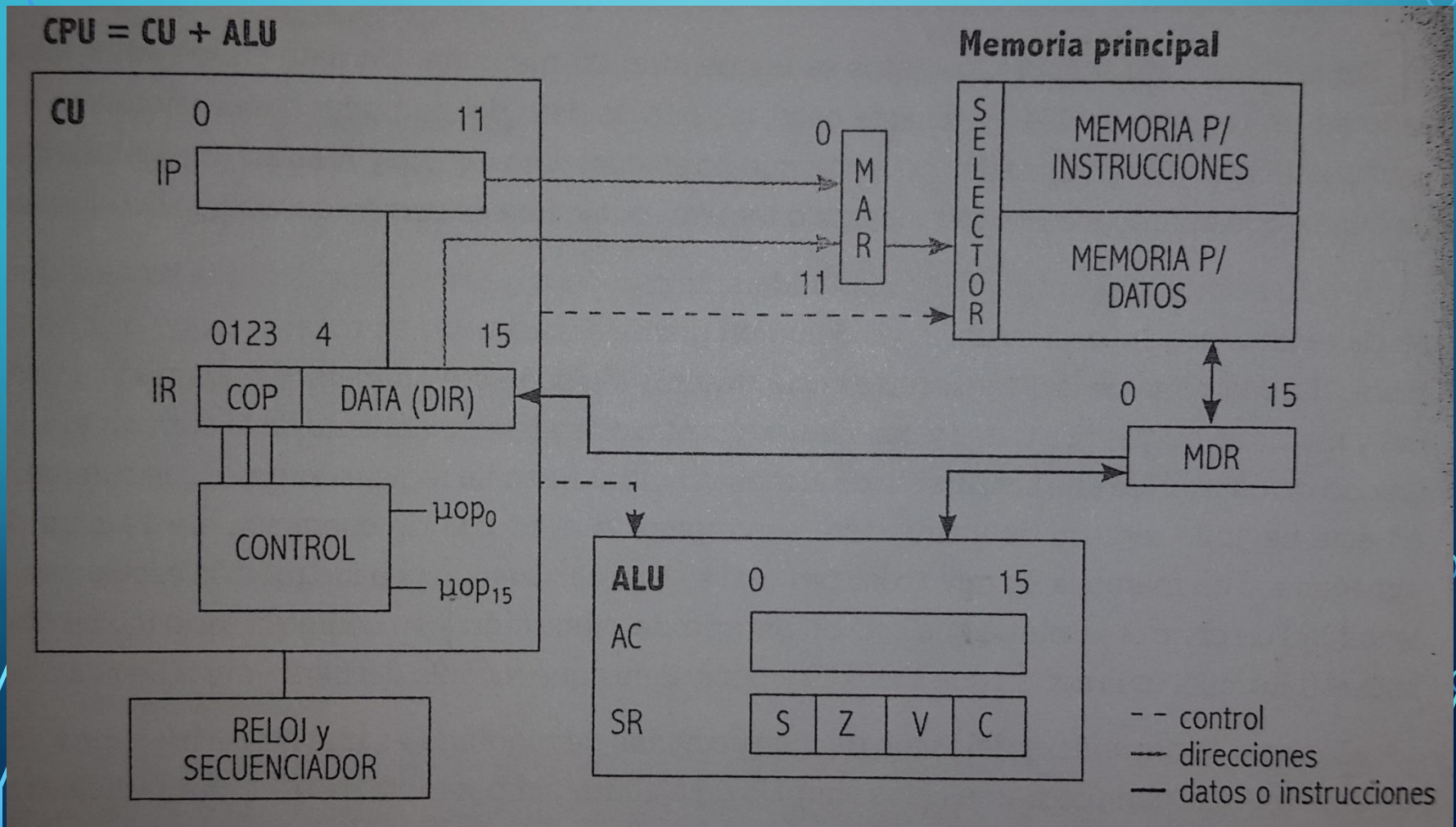
- La unidad de control lee una instrucción de la memoria, la aloja en un registro interno (registro de instrucción) e interpreta si el código de operación afecta a un dato almacenado en memoria, en cuyo caso provoca su lectura.
- Cuando una instrucción está alojada en la unidad de control se afirma que está en **estado de ejecución**.

DISEÑO DE LA COMPUTADORA

- Los diseños actuales muestran la tendencia para desarrollar la mayor cantidad de funciones sobre HW, porque esto incrementa la velocidad de procesamiento, acompañada de un declive constante de su costo.
- Hoy se implementan en HW las funciones que se utilizan en forma más frecuente y el resto se implementa en SW.

MODELO DE ESTUDIO

Computadora "1"



MODELO DE ESTUDIO

Computadora “1”

- “1” tiene un módulo de almacenamiento de lectura/escritura de 4k palabras de 16 bits cada una, con acceso aleatorio a cada palabra identificada por una dirección de 12 bits que se notará en hexadecimal, de modo que el rango de direcciones varía entre 000_H y FFF_H (0 a 4095).
- La memoria está dividida “lógicamente” en memoria asignada a programa y memoria asignada a datos.
- “1” puede ejecutar un programa por vez, o sea que su modalidad de procesamiento es la monoprogramación.

COMPUTADORA “1”

Formato de datos

- Los datos son del tipo enteros signados de 16 bits (1 para el signo y 15 para la magnitud)



COMPUTADORA “1”

Formato de instrucción

- Las instrucciones se almacenan en palabras consecutivas a partir de la palabra 000_H .
- El código de instrucción es de 16 bits, con un formato único de instrucción, el más simple, donde los primeros 4 bits definen el código de operación, y los 12 restantes definen la dirección de un dato (en memoria asignada a datos), de lo contrario, son ignorados.
- COP del 0000_B al 1111_R



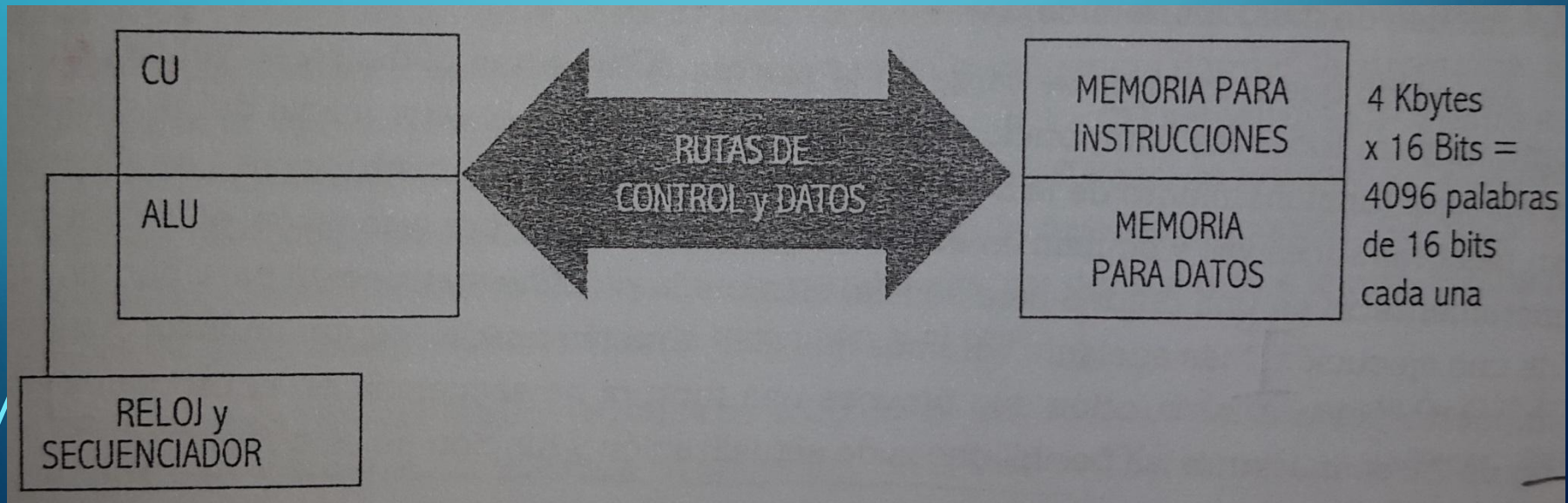
COMPUTADORA “1”

Formato de instrucción

- La **CPU** de “1” (Unidad Central de Procesamiento) se encarga de buscar las instrucciones de la memoria, interpretarlas y gestionar su ejecución, mientras que los datos son operados aritmética o lógicamente en la **ALU**.
- La función de la CPU se puede separar en dos partes:
 - Tratamiento de instrucciones
 - Operación de los datos
- Del tratamiento de instrucciones se encarga un conjunto de componentes denominado unidad de control (CU o Control Unit), sincronizado por el generador de pulsos de reloj, y de la operación de los datos se encarga la unidad aritmético-lógica (ALU o Arithmetic Logic Unit)

COMPUTADORA "1"

Formato de instrucción



COMPUTADORA “1”

CPU

- La CPU merece una dedicación especial, ya que su función es gestionar, precisamente, el procesamiento de información.
- La CU toma las instrucciones del programa almacenado en memoria para interpretarlas y ejecutarlas. Este proceso se puede dividir en las etapas siguientes:
 - Búsqueda de la instrucción en memoria
 - Interpretación del código de instrucción
 - Búsqueda del dato afectado (si afecta dato) por la instrucción

COMPUTADORA “1”

CPU – Fase *Fetch*

- Cuando la CU ejecuta cada instrucción de un programa debe alternar sus etapas *fetch* (f) y *execute* (e) desde la primera instrucción hasta la última, esto es: $f_{I_0} \cdot e_{I_0}, f_{I_1} \cdot e_{I_1}, \dots, f_{I_n} \cdot e_{I_n}$
- La secuencia del ciclo $f_{I_i} \cdot e_{I_i}$ se denomina **ciclo de instrucción**.
- Para cada fase f_{I_i} la CU debe enviar a la memoria la dirección de la palabra donde se encuentra la instrucción, una orden de lectura y una orden de transferencia de la instrucción a la CU. La CU retiene la dirección de la instrucción en un registro especial denominado “Puntero de Instrucción” (IP o *Instruction Pointer*) o “Contador de Programa” (PC o *Program Counter*)

COMPUTADORA “1”

CPU – Fase *Fetch*

- La longitud del IP depende de la cantidad de bits que se necesiten para direccionar cualquier instrucción en la memoria asignada al programa.
- * Si para “1” una instrucción I_n podría “supuestamente” alojarse en la palabra FFF_H que es la última, por lo tanto el IP mide 12 bits como máximo, que permiten 2^{12} direcciones numeradas de 0 a 4095_{10} . O sea que este puntero puede hacer referencia a las 4k direcciones de memoria, la última de ellas se representa con doce 1 binarios (FFF_H)

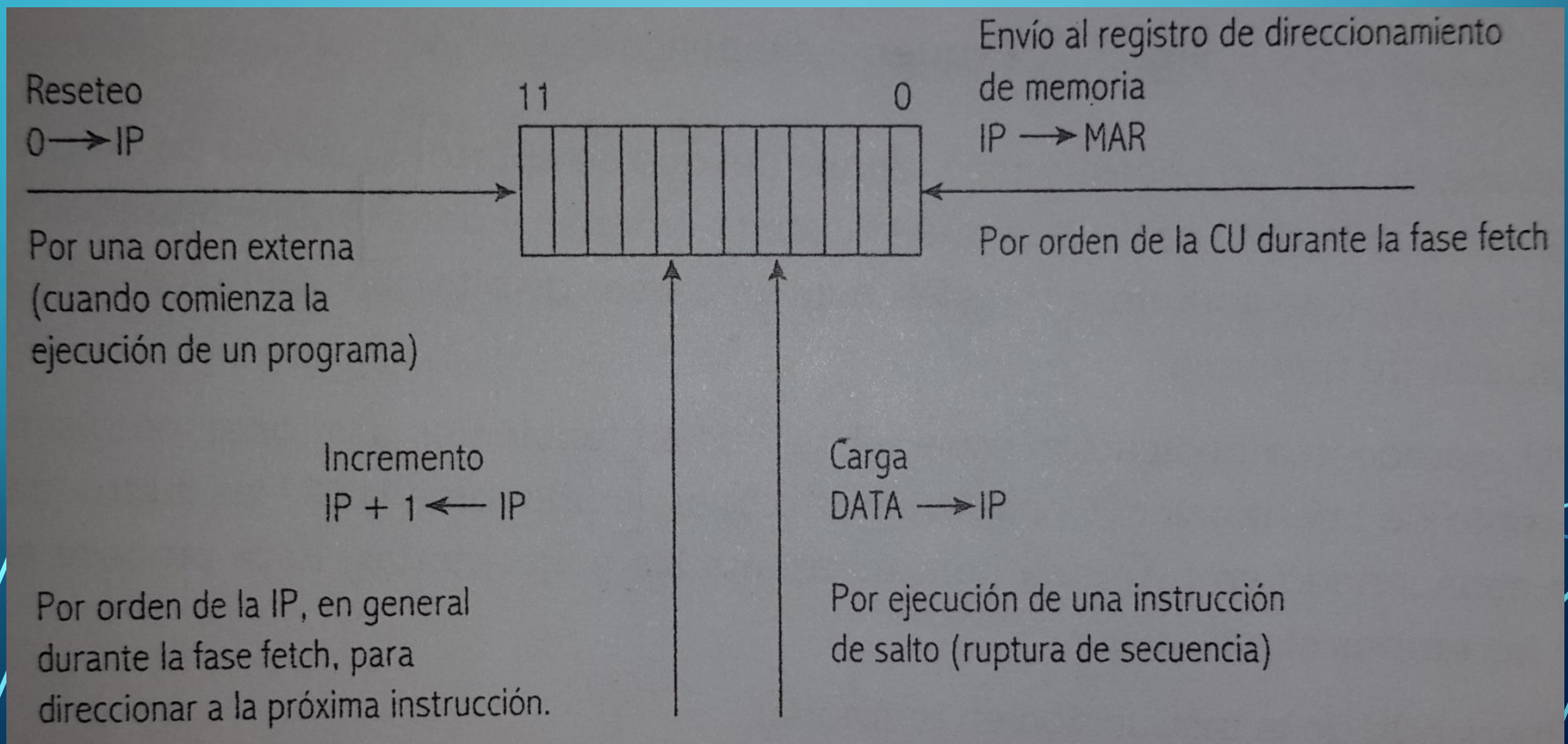
COMPUTADORA “1”

CPU – Fase *Fetch*

- Las instrucciones se almacenan en palabras sucesivas y ocupan en este caso una sola palabra, por esta razón, el IP debe poder incrementarse en una unidad para señalar siempre la próxima instrucción para buscar, luego de una ejecución.
- Debe tenerse en cuenta que el IP también puede aceptar un valor cualquiera impuesto por una instrucción que provoca una ruptura de secuencia en el programa.

COMPUTADORA "1"

CPU – Fase *Fetch*



COMPUTADORA “1”

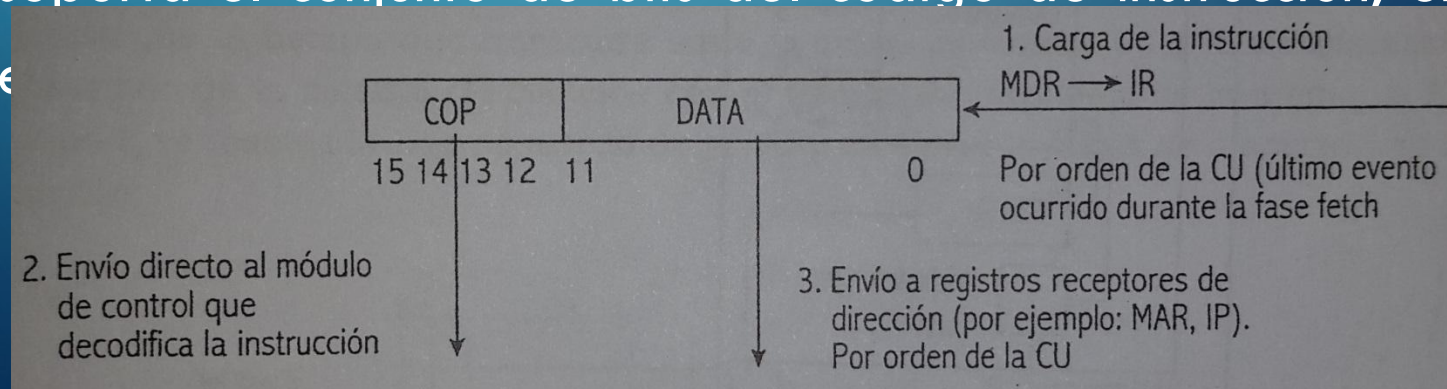
CPU – Fase *Fetch*

- Una vez que la CU envía a memoria el contenido de la IP, da la orden de lectura para que la palabra implicada, que llamaremos W, se almacene en el registro de palabra de memoria (MDR o *Memory Data Register*).
- Usualmente, la unidad de acceso a memoria es el bit; aquí la computadora “1” tiene una memoria organizada por palabras de 2 bytes y la unidad de direccionamiento es entonces la palabra (W).

COMPUTADORA “1”

CPU – Fase *Fetch*

- Por último, la etapa *fetch* termina con la transferencia de la instrucción leída a un registro interno de la CU, donde la instrucción permanecerá almacenada mientras dure su ejecución. Este registro se denomina registro de instrucción (IR o *Instruction Register*) y su capacidad soporta el conjunto de bits del código de instrucción, en



COMPUTADORA “1”

CPU – Fase *Fetch*

- En este caso, la interpretación del COP la realiza un decodificador $4.2^4 = 4.16$, lo que significa que con 4 dígitos binarios se generan 16 instrucciones referenciadas por una de las salidas del circuito COP.
- La zona denominada DATA es de 12 bits, se relaciona con el MAR (*Memory Address Register*) cuando se debe tomar un dato de memoria y con el IP cuando se debe romper la secuencia normal del programa. Esta zona se ignora cuando la instrucción no afecta dato alguno. La CU actualiza el IP y entra en la fase *execute* de la instrucción buscada.

COMPUTADORA “1”

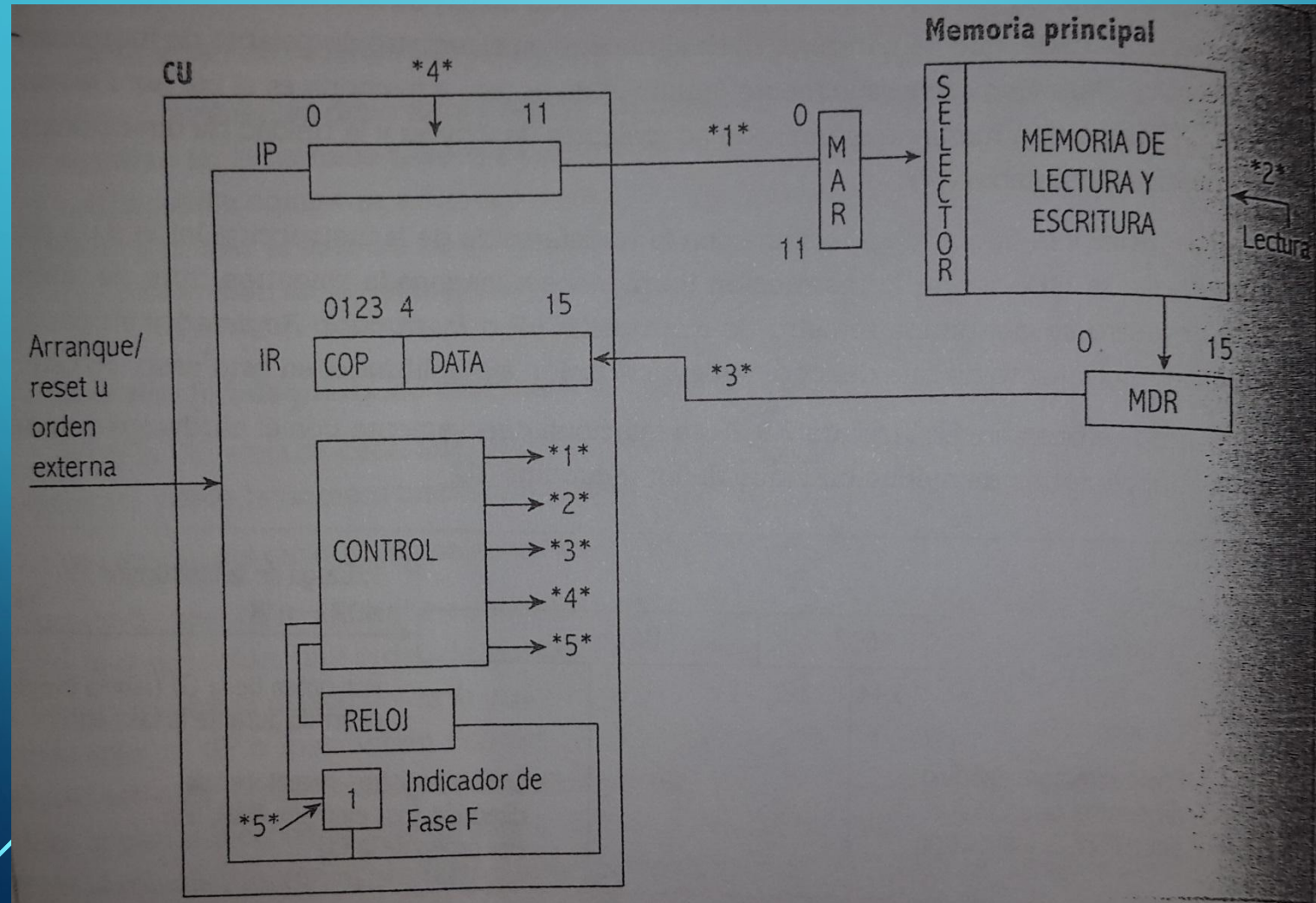
CPU – Fase *Fetch*

- En un *flag* llamado F (fase) se retiene un 1 durante la fase de búsqueda, que cambia a 0 cuando comienza la fase de ejecución. Su función es importantísima para la CU de “1”, porque le permite alternar de una fase a otra, inhibiendo órdenes de búsqueda durante la fase de ejecución, y al revés.
- Los componentes de HW implicados en la fase fetch y la secuencia de sus eventos, son los siguientes:

- | | | | |
|-----------|---|----------------------------|-------|
| 1. IP | → | MAR | t_0 |
| 2. Word | → | MDR (por orden de lectura) | t_1 |
| 3. MDR | → | IR | t_5 |
| 4. IP + 1 | → | IP | t_1 |
| 5. 0 | → | F | t_7 |

COMPUTADORA "1"

CPU – Fase *Fetch*



COMPUTADORA “1”

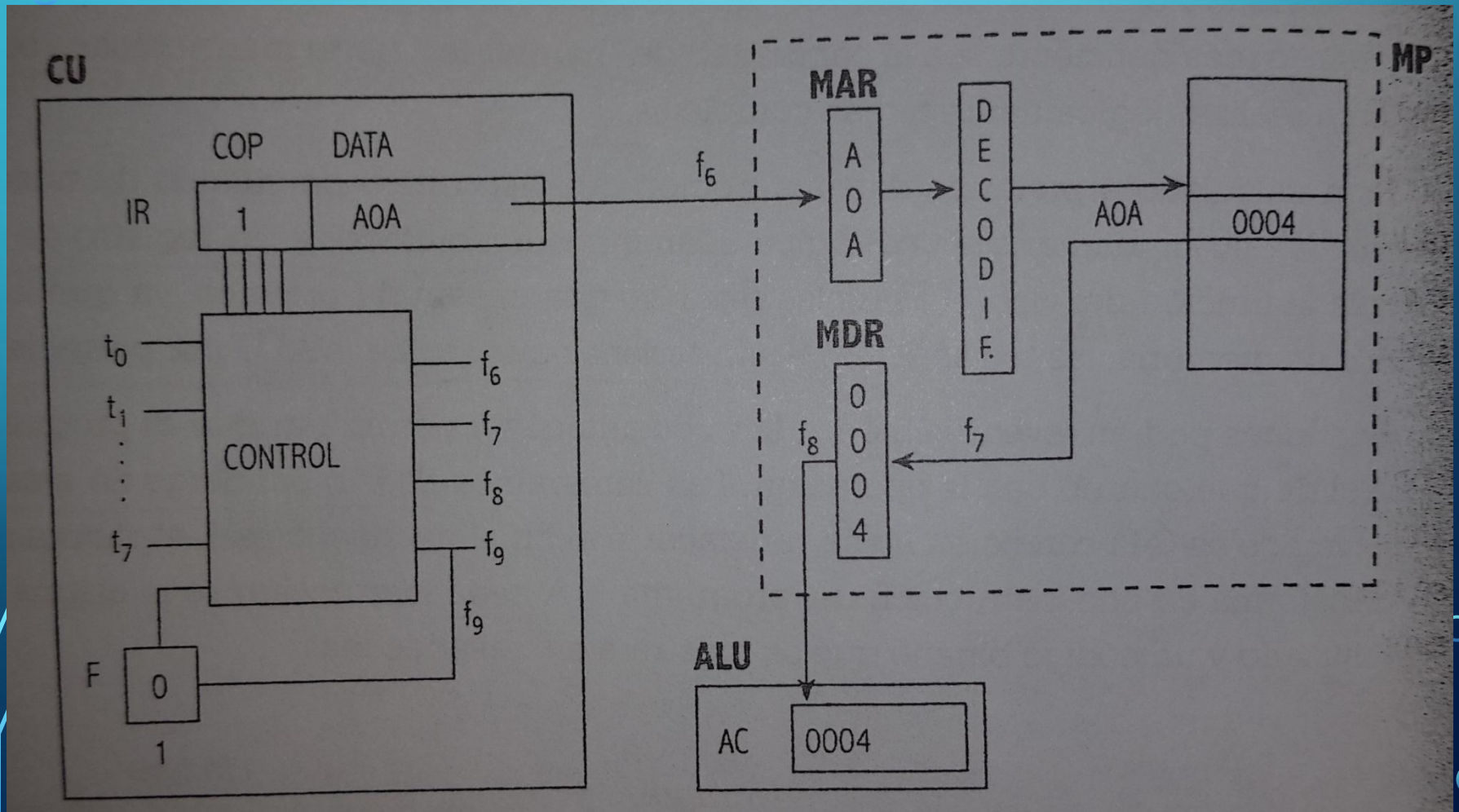
CPU – Fase *Execute*

- Una vez que se hace efectiva la búsqueda de la instrucción, la CU entra en fase de ejecución
- Ejemplo:

	Nombre	Código (representa el código de máquina)
Cargar el acumulador con la palabra A0A	LDA A0A Load accumulator	1A0A
Sumar al acumulador la palabra A0B	ADA A0B Add accumulator	2A0B
Desplazar el acumulador	SHR Shift right	AXXX
Almacenar el acumulador en la palabra A0C	STA A0C Store accumulator	3A0C
Fin del programa	HLT	0XXX

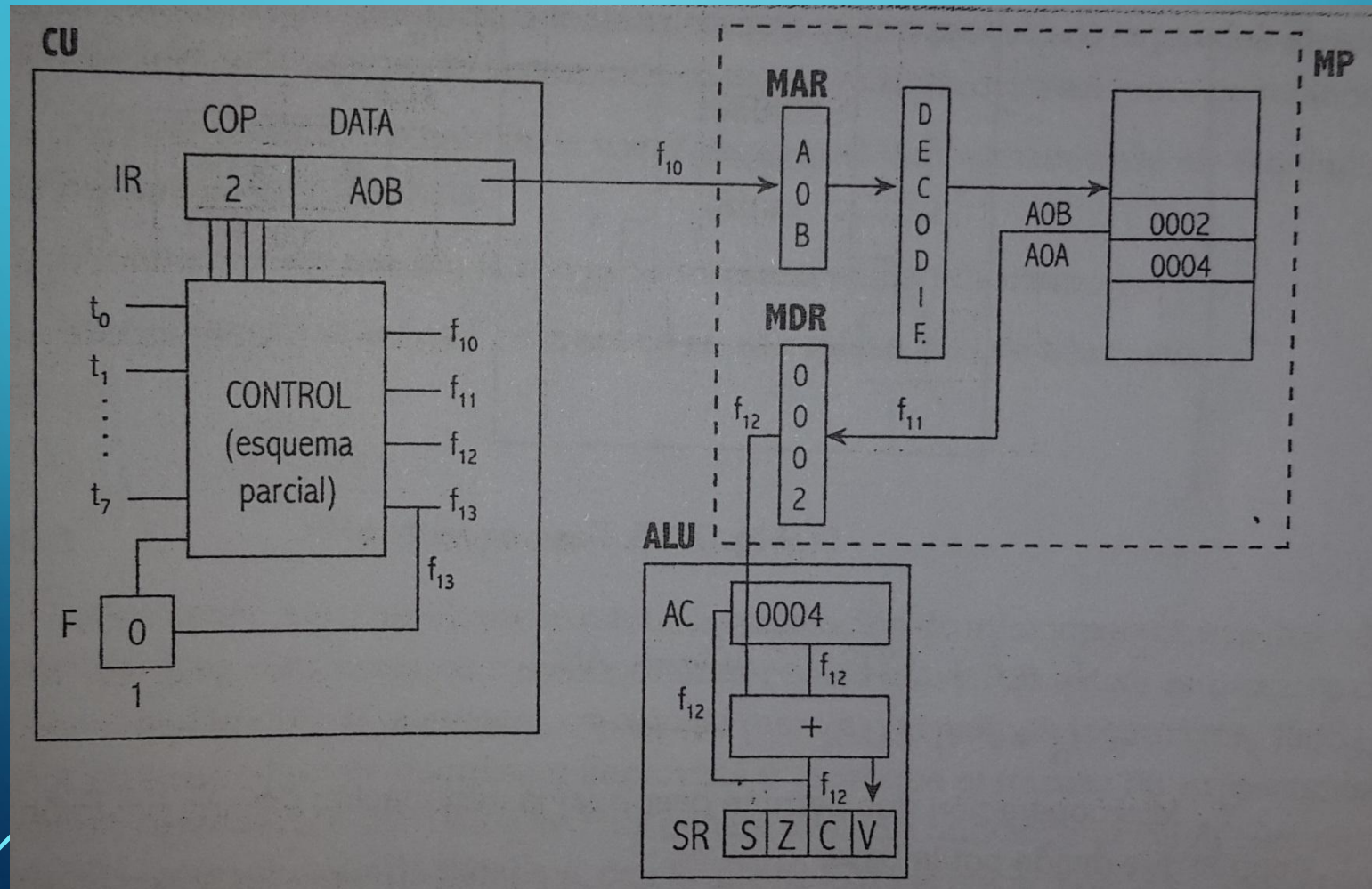
COMPUTADORA "1"

CPU – Fase *Execute* de LDA A0A



COMPUTADORA "1"

CPU – Fase *Execute* de ADA AOB



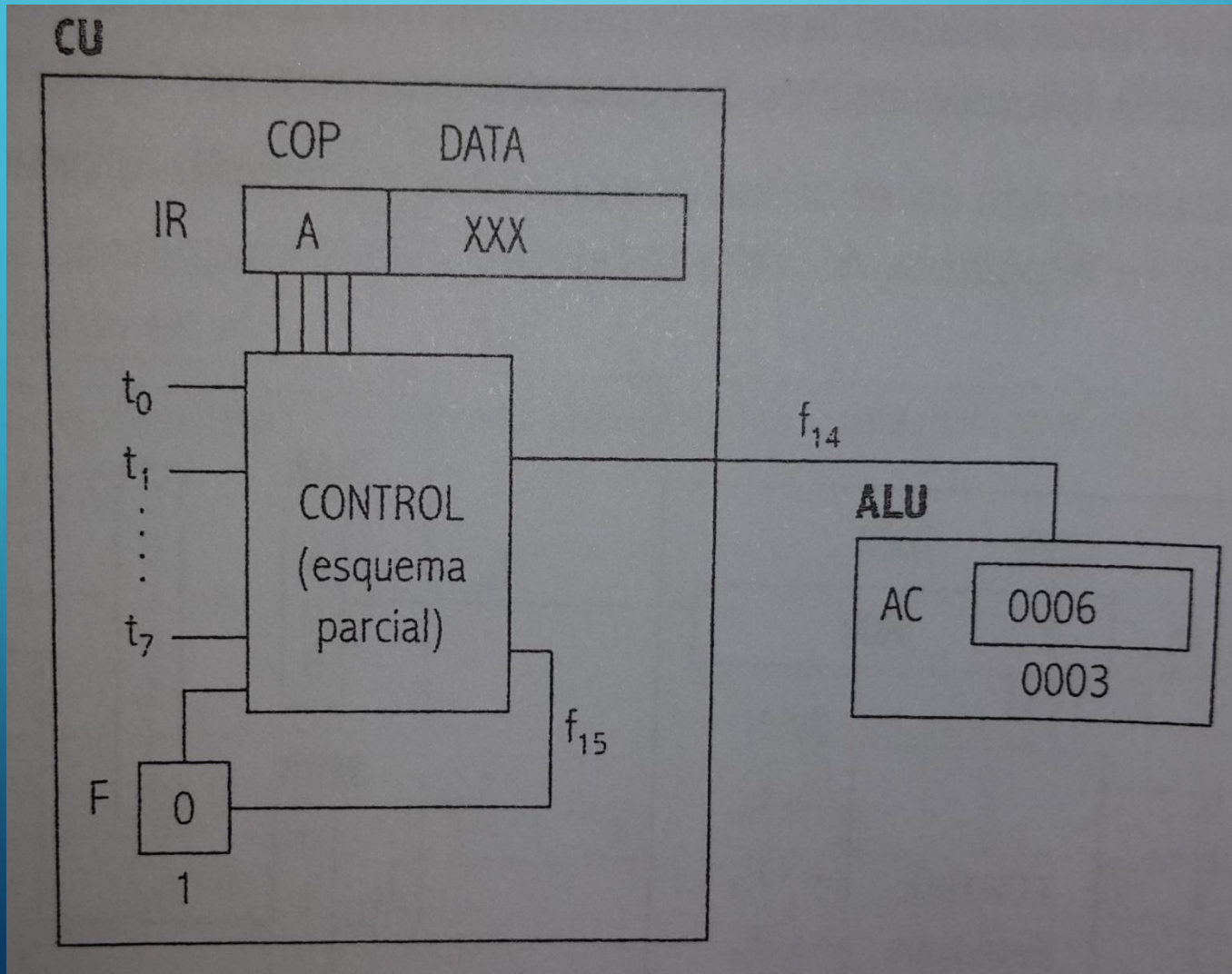
COMPUTADORA “1”

CPU – Fase *Execute* de *SHR*

- El control genera una micro-operación que ordena al acumulador desplazar su información un bit a la derecha (*shift right*).
- Incluir esta instrucción en este punto del programa implica lograr dividir el resultado almacenado en AC por 2, para calcular el promedio de los datos sumados.

COMPUTADORA "1"

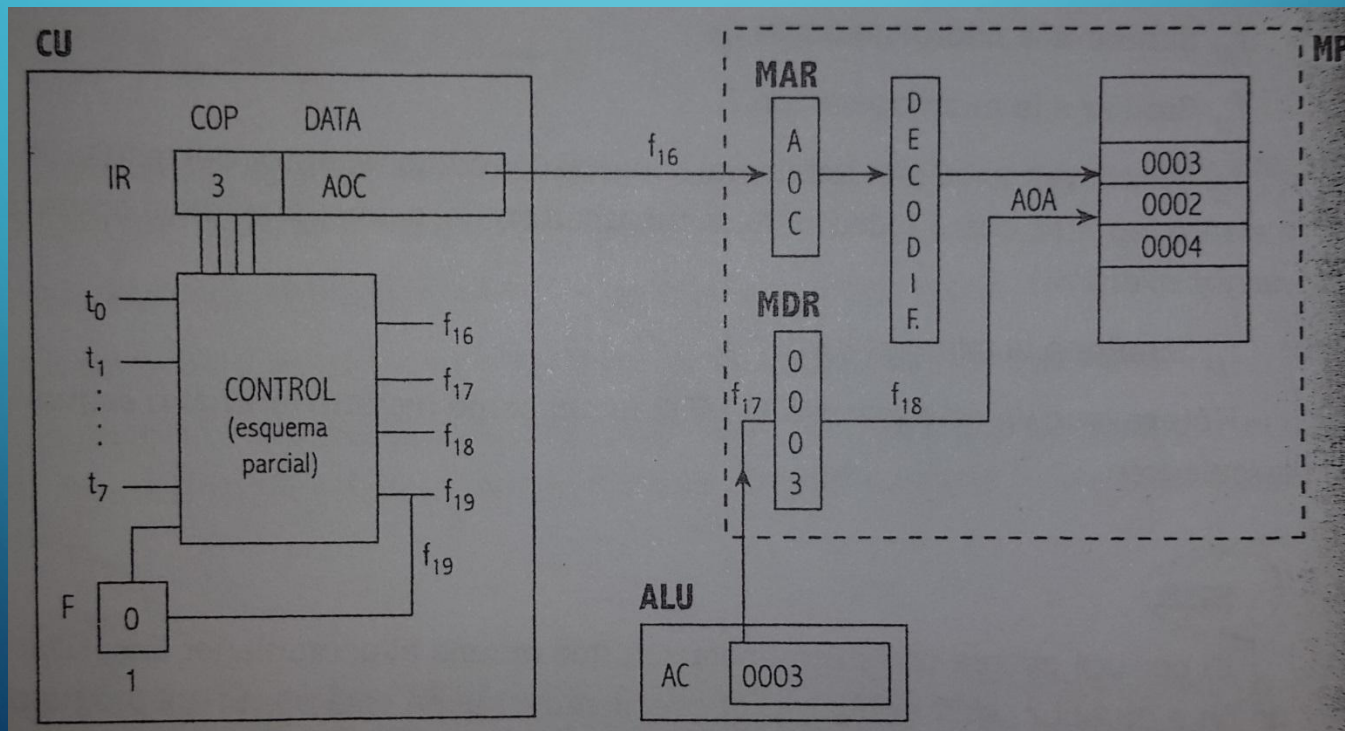
CPU – Fase *Execute* de SHR



COMPUTADORA "1"

CPU – Fase *Execute* de *STA A0C*

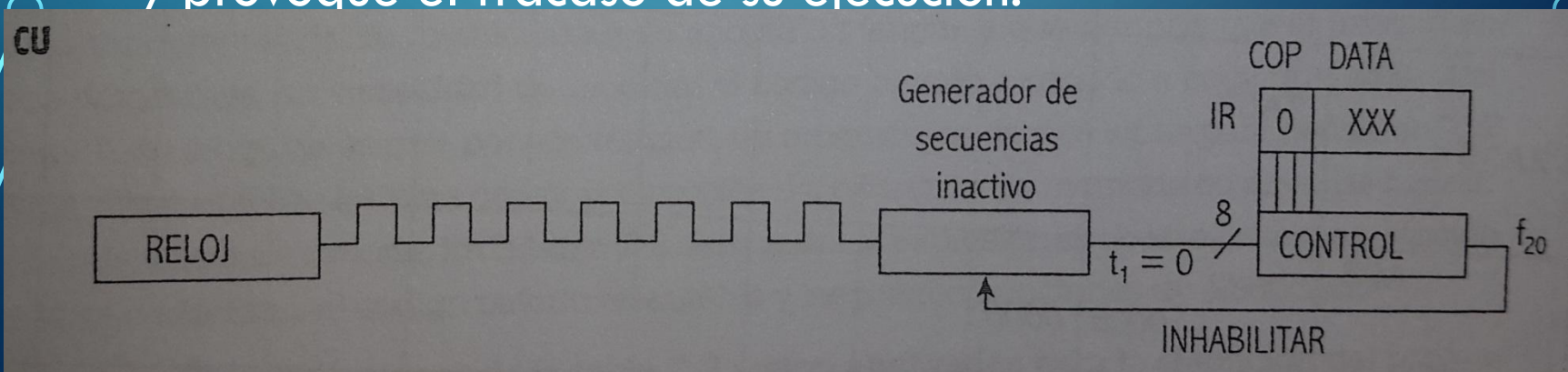
- Esta instrucción se incluyó en el programa para almacenar el resultado final en memoria



COMPUTADORA “1”

CPU – Fase *Execute* de *HLT*

- La instrucción *HLT* (*halt* = parada) inhibe la generación de micro-operaciones dejando a “1” suspendida.
- Esta instrucción es muy importante, puesto que de no existir se produce una etapa *fetch*, considerando la siguiente posición de memoria como instrucción. Esto implica que el programa continúe de manera imprevista y provoque el fracaso de su ejecución.



Esquema de evolución de ejecución de las instrucciones

Tabla 7-1. Esquema de la evolución de ejecución de las instrucciones.

[illegible]

COMPUTADORA "1"

Esquema simplificado por etapas

7.4 Presentación del modelo de estudio

Tabla 7-2. Esquema simplificado por etapas.

	<i>IP</i>	<i>IR</i>	<i>MAR</i>	<i>MDR</i>	<i>AC</i>	<i>SR</i>	<i>F</i>
Estado inicial	000	XXXX	XXX	XXXX	XXXX	x	1
Fetch	001	1A0A	000	1A0A			0
Execute LDA			A0A	0004	004		1
Fetch	002	2A0B	001	2A0B			0
Execute ADA			A0B	0002	006	0	1
Fetch	003	AXXX	002	AXXX			
Execute SHR			No afecta a mem.		003	0	1
Fetch	004	3A0C	003	3A0C		0	
Execute STA			A0C	0003		0	
Fetch	005	Instrucc de fin	004	Instrucc de fin		X	

7.4.4 Juego completo de instrucciones de "X"

Para cubrir las necesidades de programación de "X" a continuación se presentan otras instrucciones y los enunciados de algunas tareas de aplicación de ellas.

COMPUTADORA “1”

Juego completo de instrucciones de “1”

- * “1” tiene tres tipos de instrucción definidos con claridad:
 - * Las instrucciones que afectan datos almacenados en la memoria. Por ejemplo, LDA A0A.
 - * Las instrucciones que afectan datos almacenados en registros que no pertenecen a la memoria (AC, *STATUS REGISTER*, etc.). Por ejemplo, SHR.
 - * Las instrucciones que permiten la entrada de un dato a “1” desde el exterior, o la salida de un dato desde “1” al exterior. Por ejemplo, INP

COMPUTADORA “1”

Juego completo de instrucciones de “1”

* **JMP HHH**

- * Es una instrucción de salto incondicional, o sea que se utiliza cuando se necesita romper el orden secuencial del programa. HHH es la dirección de la palabra de la instrucción a la que se quiere saltar. Por eso el control transfiere directamente esta dirección al PC, de modo que la próxima etapa de búsqueda lo encuentre actualizado.

* **ANA HHH**

- * Es una instrucción que permite asociar al acumulador y al dato obtenido mediante el operando lógico AND, de modo que el contenido final del acumulador sea el resultado de la operación “producto lógico” para cada par de valores bit a bit.

* **XOA HHH**

- * Es una instrucción que permite asociar al acumulador y al dato obtenido mediante el operador lógico *OR EXCLUSIVE*, de modo que el contenido final del acumulador sea el resultado de la operación “suma exclusiva” para cada par de valores bit a bit.

COMPUTADORA “1”

Juego completo de instrucciones de “1”

- * **HLT** (*Halt*)

- * Es la instrucción que determina el fin del programa. Es imprescindible para que no se sigan buscando posiciones de memoria que no almacenan instrucciones, evitando así resultados imprevistos. “1” lo logra al inhibir la generación de señales de tiempo.

Éstas y todas las instrucciones de referencia a registro o de E/S ignoran el campo del código de instrucción DATA.

- * **CLA** (*Clear accumulator*)

- * Permite la puesta a 0 del acumulador.

- * **CMA** (*Complement accumulator*)

- * Permite la inversión de los bits del acumulador. Calcula el complemento restringido del dato almacenado en el acumulador.

COMPUTADORA “1”

Juego completo de instrucciones de “1”

- * **INC** (*Increment accumulator*)

- * Permite sumar una unidad al dato cargado en el acumulador

- * **SHR** (*Shift Right*)

- * Permite desplazar a la derecha un bit del dato almacenado en el acumulador.

- * **SNA HHH**

- * Es una instrucción de salto condicionado, que permite tomar una decisión en el programa, en función de una condición preestablecida. En este caso, la condición es que el dato almacenado en el acumulador sea negativo, esto es, que para efectuar el salto de la instrucción considere el valor 1 del flag “S” en el registro de estado.

COMPUTADORA “1”

Juego completo de instrucciones de “1”

* **SZA HHH**

- * Es una instrucción de salto condicional igual que las anteriores. En este caso, la condición que se ha de verificar es que el dato almacenado en el acumulador sea 0, por lo tanto la bandera consultada es la Z. Con $Z=1$ la condición se satisface.

* **SCA HHH**

- * Es una instrucción de salto condicional igual que las anteriores. En este caso la condición que se ha de verificar es que el último acarreo sea 1, para considerar que ésta se encuentra satisfecha.

COMPUTADORA “1”

Juego completo de instrucciones de “1”

* **INP** (*Input*)

- * Esta instrucción permite la entrada de un dato desde un periférico de entrada. La transferencia del dato se realiza entre un registro propio del periférico ENT y el acumulador. “1” está dotada de un solo periférico de entrada, por lo tanto, el campo DATA del código de instrucción no identifica direcciones de periféricos, entonces se lo ignora. El dato ingresado queda en el acumulador, por lo general, el programa lo transfiere a la memoria con una instrucción STA HHH.

* **OUT**

- * Esta instrucción permite la salida de un dato almacenado con anterioridad en el acumulador hacia un dispositivo periférico. Por lo tanto, la transferencia se realiza entre el acumulador y un registro propio del periférico SAL. “1” sólo tiene asignado un dispositivo de salida, por lo tanto, el campo DATA es ignorado.

COMPUTADORA "1"

Juego completo de instrucciones de "1"

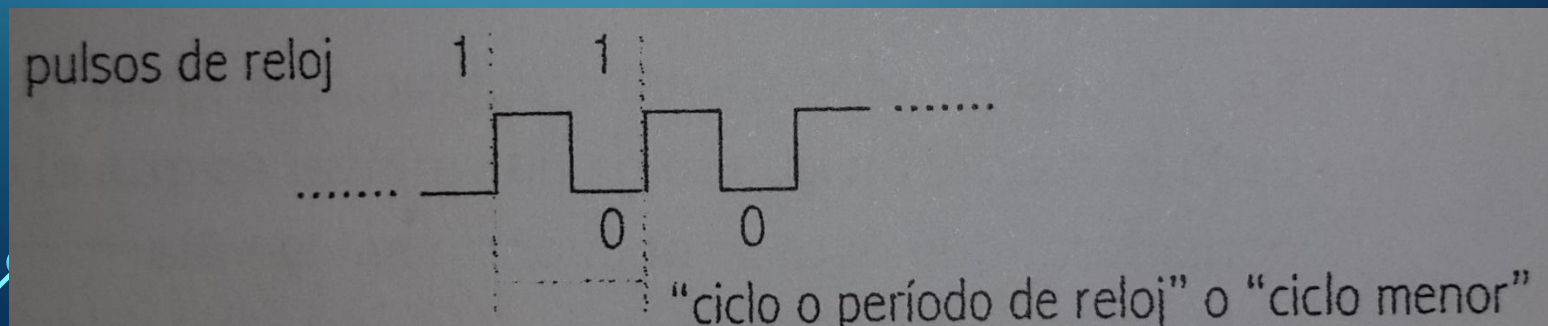
Tabla 7-3. Microoperaciones de ejecución.

	Símbolo	Taquigráfico decimal	Ejecutable binario	Microoperaciones de ejecución	
Referencia a memoria	LDA HHH	1HHH	0001bbbbbbbbbbbbbb	DATA W MDR	MAR MDR AC
	ADA HHH	2HHH	0010bbbbbbbbbbbbbb	DATA W MDR	MAR MDR AC
	STA HHH	3HHH	0011bbbbbbbbbbbbbb	DATA AC MDR	MAR MDR MW
	JMP HHH (jump)	4HHH	0100bbbbbbbbbbbbbb	DATA	IP
	ANA HHH (and acumulador)	5HHH	0101bbbbbbbbbbbbbb	DATA W MDR ^ AC	MAR MDR AC
	XOA HHH (or exclusive acumulador)	6HHH	0110bbbbbbbbbbbbbb	DATA W MDR + AC	MAR MDR AC
Referencia a registro	HLT	0XXX	0000XXXXXXXXXXXXXX	1 —————→	H
	CLA	7XXX	0111XXXXXXXXXXXXXX	0 —————→	AC
	CMA	8XXX	1000XXXXXXXXXXXXXX	AC —————→	AC
	INC	9XXX	1001XXXXXXXXXXXXXX	AC + 1 —————→	AC
	SHR	AXXX	1010XXXXXXXXXXXXXX	AC —————→	AC
	SNA	BHHH	1011bbbbbbbbbbbbbb	SI AC < 0	] —→ DATA → IP
	SZA	CHHH	1100bbbbbbbbbbbbbb	SI AC = 0	
	SCA	DHHH	1101bbbbbbbbbbbbbb	SI C ₁₅ = 1	
Entrada y salida	INP	EXXX	1110XXXXXXXXXXXXXX	ENT → AC,	0 —→ FEN
	OUT	FXXX	1111XXXXXXXXXXXXXX	AC → SAL,	0 —→ FSA

COMPUTADORA “1”

Unidad de control y sincronización del tiempo

- En la mayoría de las computadoras las señales de tiempo son generadas por un sistema de reloj. Éste se encuentra constituido por un oscilador y circuitos asociados que generan pulsos cuyo ancho y separación son determinados en forma precisa. Se denomina **ciclo de reloj** al intervalo entre 2 pulsos consecutivos de reloj.



COMPUTADORA “1”

Unidad de control y sincronización del tiempo

- La frecuencia del reloj se controla por un oscilador y cada ciclo de reloj se identifica como 1 hercio, Hz (*hertz* o ciclo). O sea $1 \text{ Hz} = 1 \text{ ciclo/segundo}$

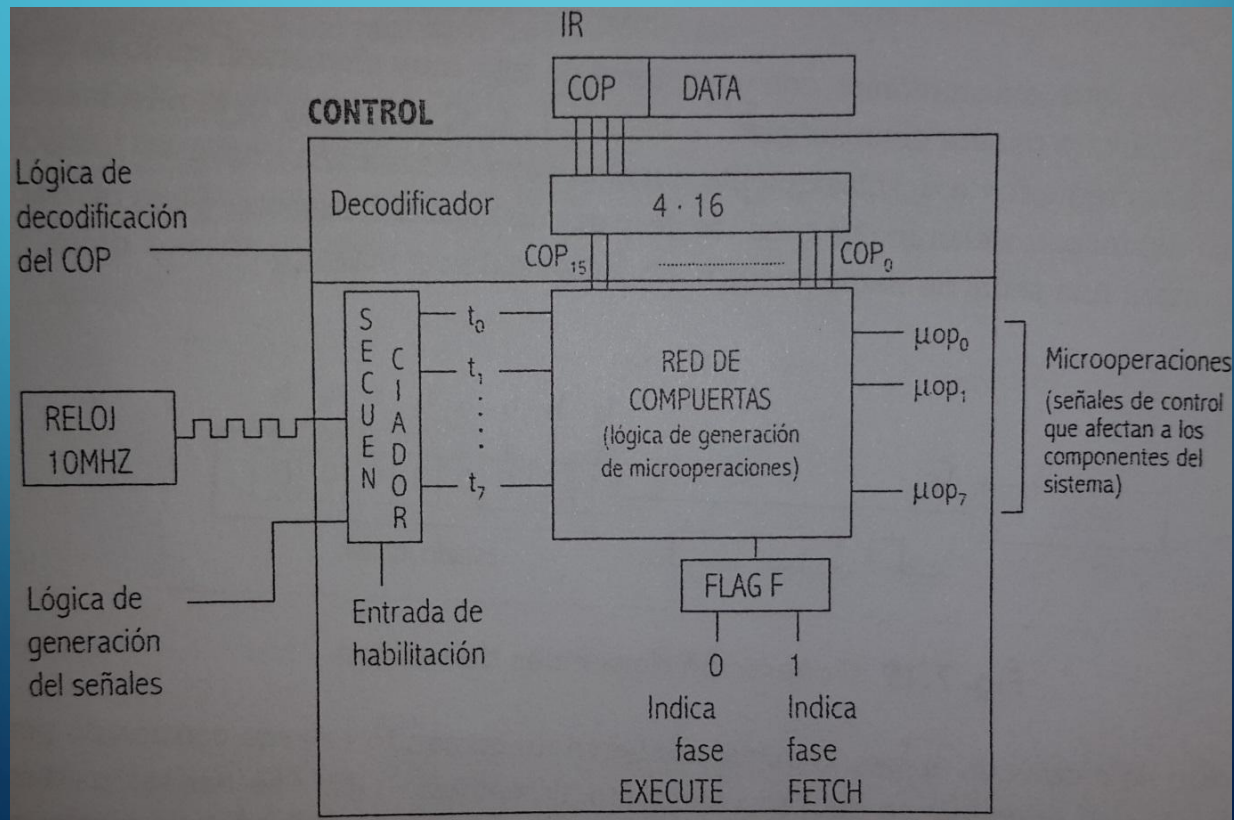
$$frecuencia = \frac{\text{cantidad de ciclos de reloj}}{\text{seg}}$$

- Las unidades de tiempo de una CPU están en orden de los nanosegundos (10^{-9} seg, ns)
- Ej.: ¿Cuántos ns “tarda” un ciclo de reloj para una computadora que trabaja con una frecuencia de 25 millones de Hz o MHz?

COMPUTADORA "1"

Unidad de control y sincronización del tiempo

- El **tiempo de acceso a memoria** es el tiempo que tarda la CU en buscar la información en la memoria y dejarla disponible en el MDR.



COMPUTADORA “1”

El módulo de cálculo: unidad aritmético-lógica

- El módulo de tratamiento de datos es la **unidad aritmético-lógica** (ALU) . En este módulo los datos se tratan según órdenes de la **unidad de control** (CU), que interpreta la instrucción durante su estado de ejecución.
- Las instrucciones de “1” que hacen referencia a la ALU son ADA, INC, CMA, SHR y SHL; ANA y XOA.

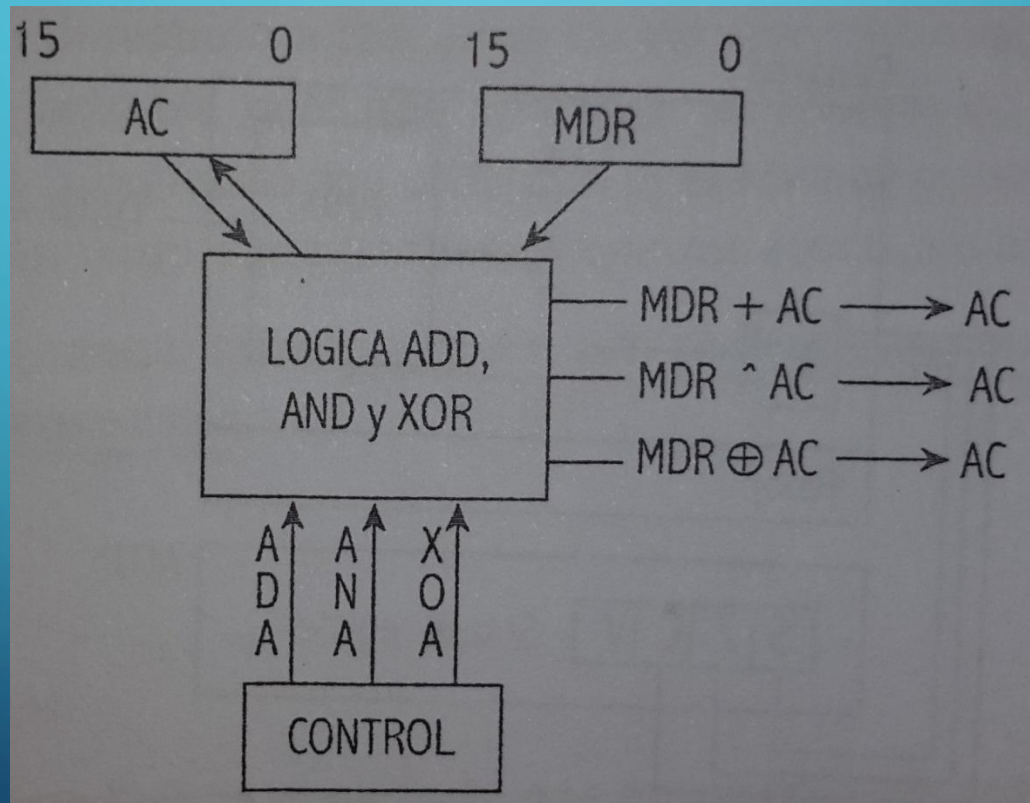
COMPUTADORA “1”

El módulo de cálculo: unidad aritmético-lógica

- La unidad aritmético-lógica de “1” está implementada en torno del registro acumulador. Cuando la operación se realiza sobre un dato, éste se almacena en el acumulador y el resultado de la operación permanece en este registro.
- Las instrucciones LDA, STA y CLA permiten el control del registro acumulador perteneciente a este módulo, gestionando su carga, descarga y puesta a 0.

COMPUTADORA "1"

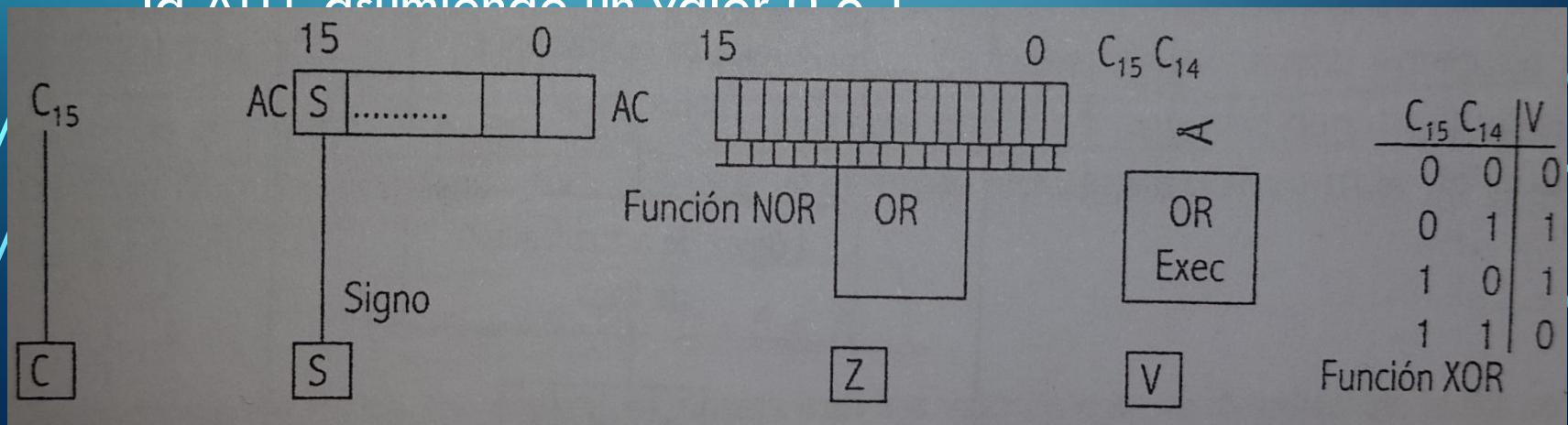
El módulo de cálculo: unidad aritmético-lógica



COMPUTADORA "1"

El módulo de cálculo: unidad aritmético-lógica

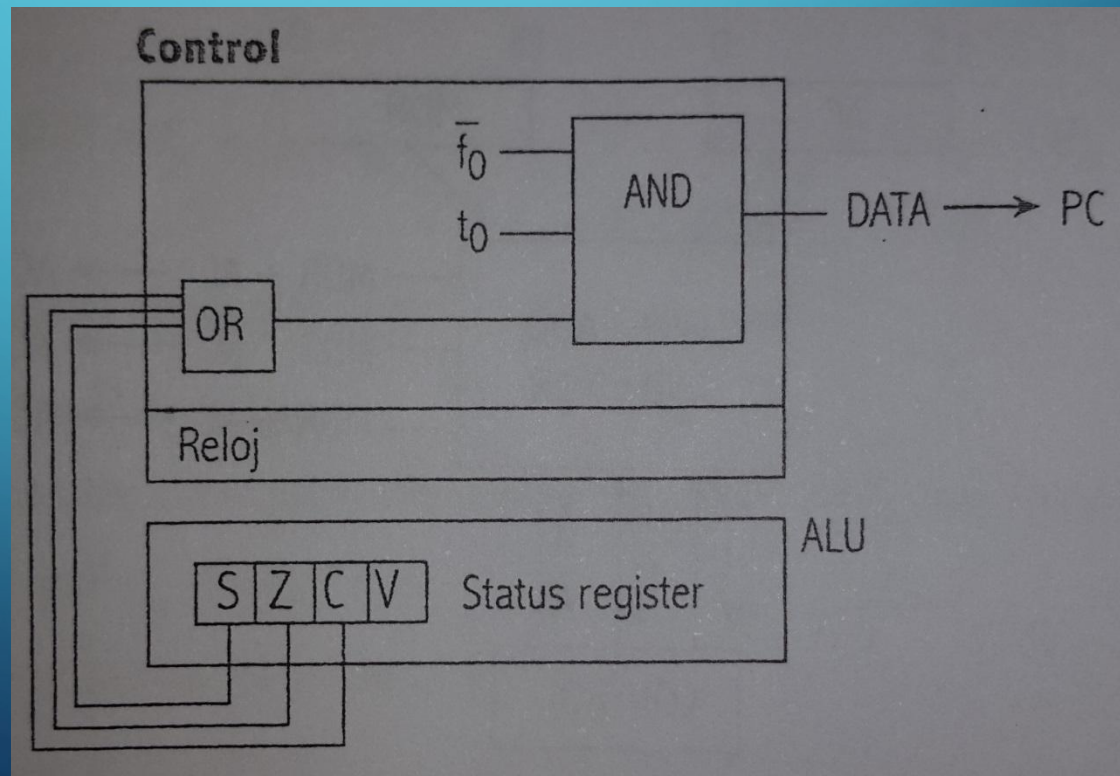
- Es importante destacar el valor de las banderas (*flags*) del **registro de estado** asociado a esta unidad (*status register* o SR). El estado de las banderas permite que el programador controle la condición final de una operación.
- Cada bandera se actualiza después de una operación en la ALU, asumiendo un valor 0 o 1.



COMPUTADORA "1"

El módulo de cálculo: unidad aritmético-lógica

- Las instrucciones de salto condicional SNA, SZA y SCA provocan salto cuando la condición de la bandera es verdadera.



COMPUTADORA “1”

Ejemplo

- Considere una porción de programa *Assembler*, donde el resultado de comparar dos números por mayor, menor e igual permite que el programador bifurque a otros puntos del programa. El método usado para la comparación es la resta de los datos:

010 LDA 3A2

011 CMA ;resta (3A1-3A2)

012 INC

013 ADA 3A1

014 SZA 100 ;si son iguales salta a 100

015 SNA 200 ;si $3A1 < 3A2$ salta a 200 (si $Z=0$ y $S=1$)

016 JMP 300 ;salta a 300

COMPUTADORA “1”

Desplazamiento de bits en los registros de la ALU

- ¿Por qué razón se desplazan bits en un registro?
Básicamente por 3 razones esenciales:
 - Para multiplicar
 - Para dividir
 - Para “testear” un bit
- En sistemas de numeración de cualquier base, el desplazamiento permite multiplicar o dividir un número por su base.
- En binario, para multiplicar un número 2 (la base) es necesario agregarle un 0 a derecha, que es equivalente a desplazar

COMPUTADORA “1”

Ejemplo

- Ej.: Hacer en binario 6.5

$$6.5 = ((6.2).2) + 6 = 30$$

	S	
	0 0000110	+6
desplazamiento	0 0001100	+12
desplazamiento	0 0011000	+24
sumo +6	0 0011110	+30

- Lo que nos da un algoritmo válido para la multiplicación, utilizando desplazamientos sucesivos y sumas

COMPUTADORA “1”

Comparación mediante resta

- Cuando se quieren comparar dos operandos, se efectúa una sustracción sin necesidad de almacenar el resultado. Las instrucciones en código de máquina permiten preguntar por la bandera indicadora de zero o de signo, de modo que
 - si $Z=0$ y $S=1$, el primer operando es menor que el segundo, y
 - Si $Z=0$ y $S=0$, el primer operando será mayor que el segundo.

COMPUTADORA “1”

Complementación

- La complementación es una operación que se aplica a un solo operando; así si el operando mide un octeto, la operación invertirá el valor de cada uno de los 8 bits

1	0	1	0	0	1	0	1
0	1	0	1	1	0	1	0

- Una de las aplicaciones tiene lugar en el cálculo de una resta, $X - Y = Z$, mediante la suma del complemento del sustraendo.

LDA Y

CMA ;complemento

INC ;suma 1

ADA X

STA Z

COMPUTADORA “1”

Otras operaciones

- El “y” lógico se usa de máscara para indicar que grupo bits del operando se va a reconocer como información.
 - Ej.: Teniendo $A=00110010$ y $B=00001111$ (como máscara), aplicando el “y” lógico permitirá que sobre Z se “copien” los primeros 4 bits de A y se anule el resto de la información. Z queda como 00000010.
- El “o” lógico permite agregar los bits de zona para transformar un dígito BCD en el correspondiente código ASCII.
 - Ej.: Teniendo $A=00001000$ y $B=00110000$, aplicando el “o” lógico quedará $Z=00111000$ ($3_{10}8_{10}$)
- El “o” exclusivo permite comparar si dos operandos son iguales, en cuyo caso el resultado es 0

COMPUTADORA “1”

El Registro de Estado asociado a la ALU

- Cada vez que la ALU realiza una operación elemental, mantiene cierta información que indica el estado final de esa operación.
- Esta información queda retenida en un “registro”, que se denomina **registro de estado** (*status register*).
- En este registro, cada bit se denomina **flag** (bandera) e indica con un “1” la condición de verdad y con “0” la condición de falsedad de un estado.

COMPUTADORA “1”

El Registro de Estado asociado a la ALU

S	Z	V	H	O	P	I	C
---	---	---	---	---	---	---	---

- La *flag* **S** o **N** es el indicador del signo del resultado.
- La *flag* **Z** es el indicador de zero.
- La *flag* **V** u **O** es el indicador de *overflow*.
- La *flag* **H** es el indicador de acarreo de los 4 primeros bits.
- La *flag* **P** es el indicador de paridad (par o impar).
- La *flag* **I** es el indicador de interrupción.